暨南大學
JINAN UNIVERSITY

# Undergraduate Lab Report

Course Title: Experiment of Computer Organization

Course No: 60080014

Student Name: 李荣耀

Student No: 2024103975

School: International School

Department:

Major: CST

Instructor: SUN Heng

Academic Year:2025~2026, Semester : 1st [√ ] 2nd[ ]

**Academic Affairs Office of Jinan University**

**Date** (dd/mm/yyyy) 2025/10/22

# Computer Organization Lab List

Student Name: Li Rongyao Student No: 2024103975

| ID | Lab Name | Type |
|---|---|---|
| 1 | Number Storage Lab | Individual |
| 2 | Manipulating Bits | Individual |
| 3 | A Simple Real-life Control System Lab | Team |
| 4 | System I/O | Individual |
| 5 | Simulating Y86-64 Program | Individual |
| 6 | Performance Lab | Team |

# Undergraduate Lab Report of Jinan University

Course Title Experiment of computer organization

Evaluation

Lab Name Manipulating Bits Instructor Sun Heng

Lab Address Room 116

Student Name Li Rongyao

Student No 2024103975

College International School

Department Major CST

Date 2025/10/22 Afternoon

## 1. Introduction

The purpose of this lab is to become more familiar with bit-level representations and arithmetic of integers in chapter 2. To do this by solving a series of programming puzzles.

## 2. Lab Instructions or Steps

Start by copying bits.c to a directory on a Linux machine in which to do the work through modifying the file.

The bits.c file contains a skeleton for each of the 5 programming puzzles. The assignment is to complete each function skeleton using only straightline code for the integer puzzles (i.e., no loops or conditionals) and a limited number of C arithmetic and logical operators. Specifically, it is only allowed to use the following eight operators:

! ~ & ^ | + << >>

3. **Lab Device or Environment**

   Ubuntu 16.04 (64-bit) on virtual machine (VMware workstation 17 Pro)

4. **Results and Analysis**

   The result snapshot has been pasted in Appendix.

   Analysis:

   1. pow2plus4():

      First, we exploit the x-bit left shifts to represent 2 power x, then plus 4 to obtain the result.

   2. bitAnd()

      By De Morgan's Law, we can know x & y == ~(~x | ~y). So we can directly obtain the result by the formula.

   3. getByte()

      To get the n-th byte of x, we should first move the target byte to the least significant bits. The bits of moving is n*8, that is n << 3. Then we can use 0xff, which is all zero except the last byte. If we take bit AND of the moved target and 0xff, we can get the corresponding byte.

   4. negate()

      By the rule of two's complement's negation, the negation of an integer needs to reverse all bit (0 to 1, 1 to 0) then plus 1.

We can obtain the result by the formula.

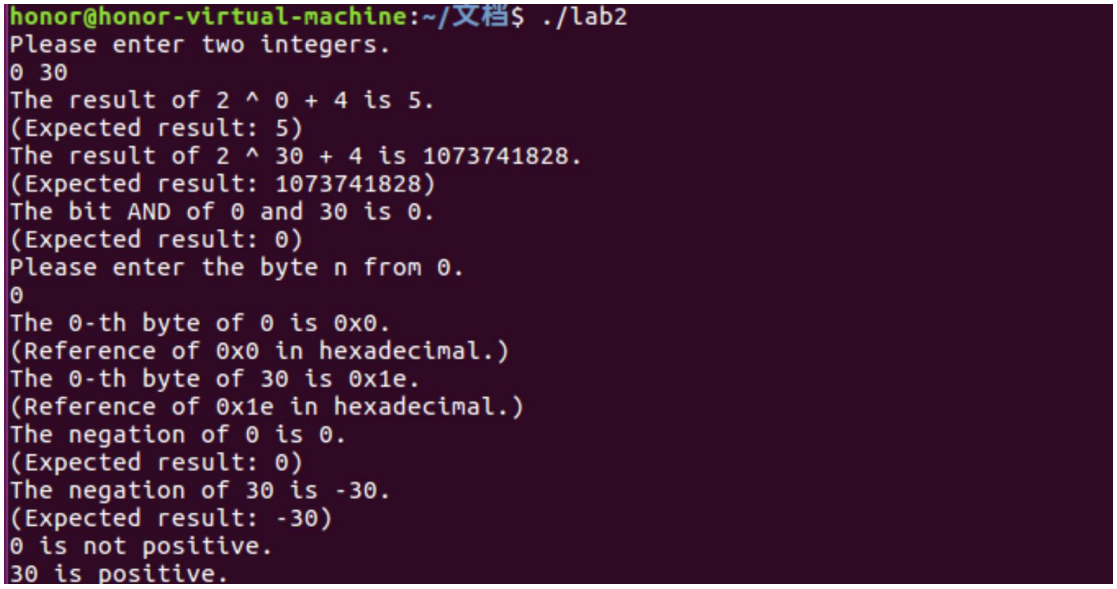
5. isPositive()

   To judge whether an integer is positive, we just need to judge the sign digit, the most significant bit. So we can do arithmetical right shift until all bits are the same as the sign digit. If the target is 0, take negation to get TRUE. If not, take negation to get FALSE.

   But for integer 0, we need to do extra judgment due to its sign digit is 0 but it is not positive. So we can take OR of the former result and 0 to get the final result.

## 5. Appendix (Program Code and Snapshot)

Result Snapshot:




Source code:

```
#include <stdio.h> //for standard input and output in main() to test
/*
 * STEP 1: Read the following instructions carefully.
 */
```

```
/* CODING RULES:
 *
 * Replace the "return" statement in each function with one
 * or more lines of C code that implements the function. Your code
 * must conform to the following style:
 *
 * int Funct(arg1, arg2, ...) {
 *     /* brief description of how your implementation works *
 *     int var1 = Expr1;
 *     ...
 *     int varM = ExprM;
 *
 *     varJ = ExprJ;
 *     ...
 *     varN = ExprN;
 *     return ExprR;
 * }
 *
 * Each "Expr" is an expression using ONLY the following:
 * 1. Integer constants 0 through 255 (0xFF), inclusive. You are
 *     not allowed to use big constants such as 0xffffffff.
 * 2. Function arguments and local variables (no global variables).
 * 3. Unary integer operations ! ~
 * 4. Binary integer operations & ^ | + << >>
 *
 * Some of the problems restrict the set of allowed operators even
further.
 * Each "Expr" may consist of multiple operators. You are not restricted
to
 * one operator per line.
 *
 * You are expressly forbidden to:
 * 1. Use any control constructs such as if, do, while, for, switch, etc.
 * 2. Define or use any macros.
 * 3. Define any additional functions in this file.
 * 4. Call any functions.
 * 5. Use any other operations, such as &&, ||, -, or ?:
 * 6. Use any data type other than int.  This implies that you
 *    cannot use arrays, structs, or unions.
 *
 *
 * You may assume that your machine:
 * 1. Performs right shifts arithmetically.
```

```
 * 2. Has unpredictable behavior when shifting an integer by more
 *    than the word size.
 * 3. Uses 32-bit representations of integers.
 */

/*EXAMPLES OF ACCEPTABLE CODING STYLE:
 *   pow2plus1 - returns 2^x + 1, where 0 <= x <= 31
*/
int pow2plus1(int x) {
    /* exploit ability of shifts to compute powers of 2 */
    return (1 << x) + 1;
}
 /* STEP 2: Modify the following functions according the coding rules.
 */

/*
 * pow2plus4 - returns 2^x + 4, where 0 <= x <= 31
 *   Legal ops: + <<
 */
int pow2plus4(int x){
    /*exploit the shifts to express*/
    return (1 << x) + 4;
}

/*
 * bitAnd - x&y using only ~ and |
 *   Example: bitAnd(6, 5) = 4
 *   Legal ops: ~ |
 */
int bitAnd(int x, int y){
    /*Use De Morgan's Law, x & y == ~(~x | ~y)*/
    return ~(~x | ~y);
}

/*
 * getByte - Extract byte n from word x
 *   Bytes numbered from 0 (LSB) to 3 (MSB)
 *   Examples: getByte(0x12345678,1) = 0x56
 *   Legal ops: & << >>
 */
int getByte(int x, int n){
    /*First move the target byte to LSB, then use 0xff to obtain the byte*/
    return (x >> (n << 3)) & 0xff;
}
```

```c
/*
 * negate - return -x
 *   Example: negate(1) = -1.
 *   Legal ops: ~ +
 */
int negate(int x){
    /*The rule of two's complement's negation*/
    return (~x + 1);
}

/*
 * isPositive - return 1 if x > 0, return 0 otherwise
 *   Example: isPositive(-1) = 0.
 *   Legal ops: ! | >>
 */
int isPositive(int x){
    /*First judge the sign digit, then eliminate 0 and get negation to
obtain final result*/
    return !((x >> 31) | (!x));
}

//through main function to test the above functions
int main(){
    int x, y, n;
    printf("Please enter two integers.\n");
    scanf("%d%d", &x, &y);

    /*pow2plus4*/
    printf("The result of 2 ^ %d + 4 is %d.\n", x, pow2plus4(x));
    int power = 1;
    int i = 1;
    while (i <= x){
        power *= 2;
        i++;
    }
    printf("(Expected result: %d)\n", power + 4);
    printf("The result of 2 ^ %d + 4 is %d.\n", y, pow2plus4(y));
    power = 1;
    i = 1;
    while (i <= y){
        power *= 2;
        i++;
    }
```

```c
    printf("(Expected result: %d)\n", power + 4);

    /*bitAnd*/
    printf("The bit AND of %d and %d is %d.\n", x, y, bitAnd(x, y));
    printf("(Expected result: %d)\n", x & y);

    /*getByte*/
    printf("Please enter the byte n from %d.\n", x);
    scanf("%d", &n);
    printf("The %d-th byte of %d is 0x%x.\n", n, x, getByte(x, n));
    printf("(Reference of 0x%x in hexadecimal.)\n", x);
    printf("The %d-th byte of %d is 0x%x.\n", n, y, getByte(y, n));
    printf("(Reference of 0x%x in hexadecimal.)\n", y);

    /*negate*/
    printf("The negation of %d is %d.\n", x, -x);
    printf("(Expected result: %d)\n", -x);
    printf("The negation of %d is %d.\n", y, -y);
    printf("(Expected result: %d)\n", -y);

    /*isPositive*/
    if (isPositive(x)){
        printf("%d is positive.\n", x);
    }else{
        printf("%d is not positive.\n", x);
    }
    if (isPositive(y)){
        printf("%d is positive.\n", y);
    }else{
        printf("%d is not positive.\n", y);
    }

    return 0;
}
```